

CYBR 437: Secure Coding

Static Analysis Tools for C

Instructor: Abinash Borah

Outline

- Static Analysis Tools for C
- GCC Static Analyzer
- Some Other Open-Source Tools
- Examples

Static Analysis Tools for C

- Various static analysis tools are available for C (as well as for lots of other languages)
- NIST maintains a list of 'Source Code Security Analyzers' for different languages here:
<https://www.nist.gov/itl/ssd/software-quality-group/source-code-security-analyzers>
- These tools include both open-source and commercial tools
- Open-source tools are freely available and widely used
- Commercial tools typically offer advanced features, dedicated support, and certifications for industry compliance standards

GCC Static Analyzer

- GCC includes a built-in static analyzer, which can be enabled using the `-fanalyzer` flag

Example: `gcc -fanalyzer example.c -o example`

- First implementation was included in GCC 10, with major changes coming in GCC 11
- Focused on finding logic and memory errors that normal compiler warnings might miss
- `-fanalyzer` option enables a static analysis of program flow that looks for “interesting” interprocedural paths through the code, and issues warnings for problems found on them
- This analysis is much more expensive than other GCC warnings

GCC Static Analyzer (contd.)

- Unlike standard warnings (-Wall, -Wextra) that look at individual statements, the static analyzer:
 - Tracks execution paths across functions
 - Simulates program flow
 - Models memory and state changes
 - Detects resource leaks and misuse
 - Reports potential undefined behavior
- Described as a lightweight built-in alternative to other advanced tools

GCC Static Analyzer (contd.)

- Enabling `-fanalyzer` effectively enables these warnings →
- If `-fanalyzer` is enabled, the analyzer uses various heuristics to explore the control flow and data flow in the program, but these can be defeated by sufficiently complicated code
- By default, the analysis silently stops if the code is too complicated for the analyzer to fully explore and it reaches an internal limit
- `-Wanalyzer-too-complex` option warns if this happens

```
-Wanalyzer-double-fclose  
-Wanalyzer-double-free  
-Wanalyzer-exposure-through-output-file  
-Wanalyzer-file-leak  
-Wanalyzer-free-of-non-heap  
-Wanalyzer-malloc-leak  
-Wanalyzer-possible-null-argument  
-Wanalyzer-possible-null-dereference  
-Wanalyzer-null-argument  
-Wanalyzer-null-dereference  
-Wanalyzer-stale-setjmp-buffer  
-Wanalyzer-tainted-array-index  
-Wanalyzer-unsafe-call-within-signal-handler  
-Wanalyzer-use-after-free  
-Wanalyzer-use-of-pointer-in-stale-stack-frame
```

GCC Static Analyzer (contd.)

- GCC static analyzer offers detailed diagnostics, including:
 - Where a variable is allocated
 - How control flows
 - Where the bug occurs, etc.
- Limitations:
 - Primarily focused on C with limited C++ support
 - Slower than normal compilation
 - May produce false positives in complex code
 - Not as deep as dedicated commercial static analyzers

Cppcheck

- Cppcheck is an open-source static analysis tool for C/C++
- Offers code analysis to detect bugs and focuses on detecting undefined behavior and dangerous coding constructs
- The goal is to have very few false positives (incorrect warnings)
- Cppcheck is available both as open-source and as a commercial version
- Can be installed with: `sudo apt install cppcheck`
- Example use: `cppcheck --enable=all --inconclusive example.c`

Cppcheck (contd.)

- The option: `--enable=<id>` enables additional checks
- Available ids are:
 - `all` - Enable all checks
 - `style` - Check coding style
 - `performance` - Enable performance messages
 - `portability` - Enable portability messages
 - `information` - Enable information messages
 - `unusedFunction` - Check for unused functions
 - `missingInclude` - Warn if there are missing includes
- By default, none of the additional checks are enabled
- Several ids can be provided separated by commas, e.g. `--enable=style,unusedFunction`
- `--inconclusive` option allows that Cppcheck reports even though the analysis is inconclusive

Splint

- Another open-source static analysis tool for C
- Offers detection of bugs, security vulnerabilities, and design inconsistencies
- Allows annotations (special comments) to describe intended program behavior

Example: `/*@null@*/ int *fun (int num);` (means the function may return null)

- Performs flow-sensitive analysis (tracks state changes through code paths)
- Not actively maintained compared to modern tools
- Can be installed with: `sudo apt install splint`
- Example use: `splint example.c`

Clang Static Analyzer

- A powerful path-sensitive static analysis tool for C, C++, and Objective-C
- It focuses on logic and runtime bugs
- Symbolically executes a program and explores possible execution paths to find problems
- Actively maintained and hence offers excellent support for modern C/C++
- Can be installed with: `sudo apt install clang-tools`
- Example use: `scan-build gcc example.c -o example`

Flawfinder

- Another open-source static analysis tool for C/C++
- Lightweight, fast, and focused specifically on security risk detection, rather than full program correctness
- Flawfinder performs pattern-based static analysis
- Assigns a risk level to the vulnerabilities found
- Can be installed with: `sudo apt install flawfinder`
- Example use: `flawfinder example.c`